

Restate vs Temporal: what the next size costs (v2)

Audience: teams running Temporal today and comfortable with it. This paper meets that team where it stands and asks what each engine charges when workflows get longer and releases keep shipping.

1. Your team runs Temporal comfortably, and the workflows keep getting longer.

Temporal earned that comfort. Your workers poll task queues (a pull loop each worker owns), the cluster splits duty across four named services (frontend, history, matching, worker), and state lands in an external database (often PostgreSQL or Cassandra) with a separate visibility store (often Elasticsearch) beside it. That shape is legible. On-call knows which dashboard to open.

The trouble starts with success. A pilot with twelve steps becomes an onboarding flow with forty. Then it absorbs billing. Then it absorbs provisioning. One team we describe here runs, for example, a fulfillment workflow that lives for months and touches six downstream systems before it completes.

Nothing breaks at this point. That is the point. The engine holds short histories well, which means longer histories arrive as a reward for doing good work, not as a mistake anyone made.

Comfort has its own evidence. Deployments go out weekly. Alerts stay quiet. New hires learn the model (workflows, activities, workers) in days and ship in their first sprint. A team with that record has no reason to shop for engines, and this paper gives none. The argument starts from trust earned, then asks what that trust costs at the next order of magnitude, which means the reader stays the expert on their own system throughout.

Small histories hide the price of the architecture. Every event appends to a per-workflow history the cluster must load, append, and replicate through the persistence backend (the database behind the history service). Short runs never notice. Runs that live for months do.

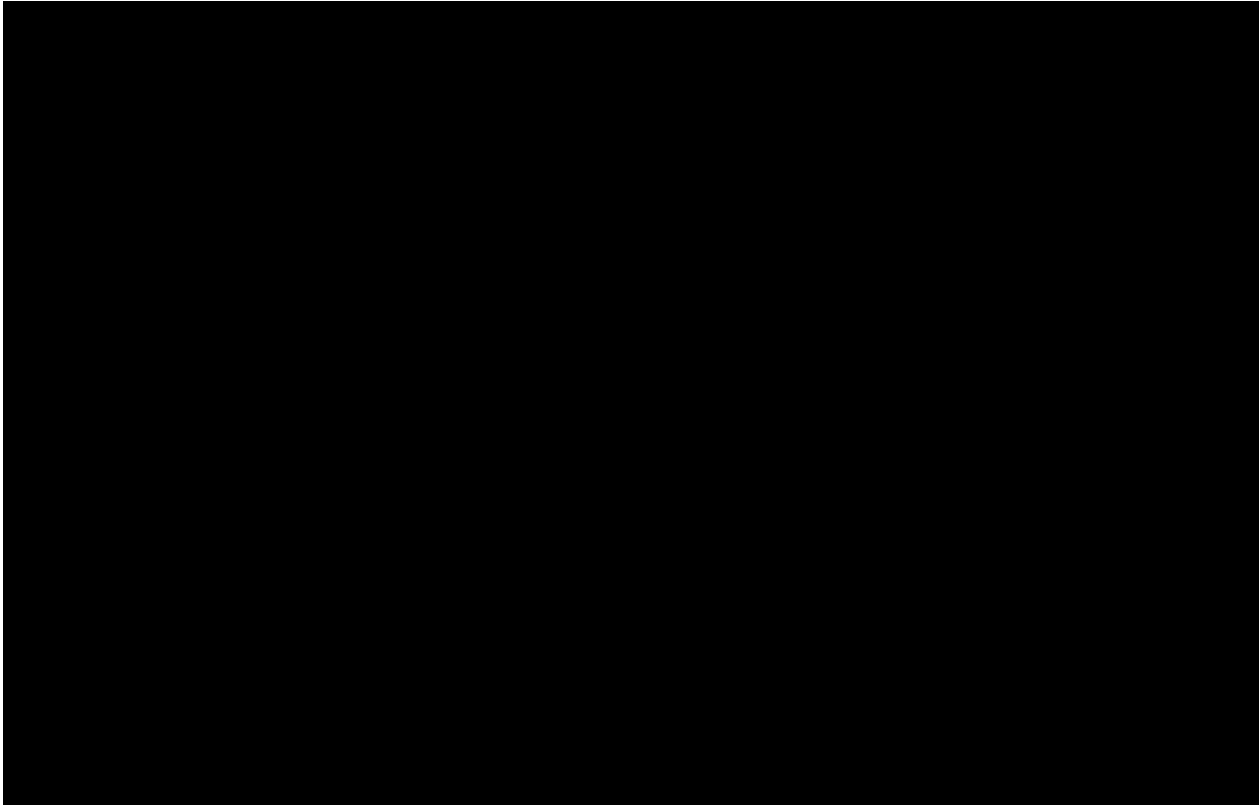


Figure 1: Length is the tax base: per-workflow history cost grows with workflow lifetime.

So length is the tax base this whole paper measures¹. The question is never whether Temporal runs your current workload. It runs it. The question is what each engine charges when that workload doubles in length and ships another release mid-flight, which means the rest of this paper reads as a bill, not a review.

2. Every release that must not break a running workflow ships version branches inside the workflow code, so the code carries its own release history.

Workflow code is durable. A deployment made in March still governs a workflow that started in March, even after the team ships again in June. Temporal names this obligation determinism (workflow code must replay the same path), and the patching API (getVersion and its patch markers) is the tool teams reach for first.

The cost is visible in the source. Temporal's own blog on worker versioning says patching "adds clutter to your Workflow source files over time"². Each marker stays because some old execution may still replay it. For example, a file that began as forty clean lines carries three version branches a year later, each with its own comment explaining which quarter it belongs to.

Worker versioning is the newer infra-level answer (build IDs and version sets assigned at deployment rather than markers in code). It moves the decision out of the workflow file and onto the fleet. Teams with strict release discipline report that it helps. It does not delete the markers already committed, which means adoption cleans the next file faster than it cleans this one.

The deeper charge is attention. Someone must remember which branch guards which generation of running executions, and no compiler checks that map. Delete a marker one quarter early and a replaying workflow fails at 2 a.m. Keep it forever and every reader pays.

Reviews get slower too. A reviewer opening a workflow file with, for example, five markers must reconstruct five generations of intent before judging the sixth change. Tests multiply for the same reason: each branch wants coverage, and the suite grows with the history it guards. Short files stay reviewable. Museum files get skimmed, which is where the next incident starts.

That memory load compounds with each release, so the workflow file becomes a museum of its own deploys, which means the team ships new logic slower the longer the workflow lives.

3. The shard count that sets your scale cannot change without a database reset, which means growth replays a migration, not a config edit.

History in Temporal is sharded (partitioned across a fixed shard count set at cluster creation; the default most teams meet first is 512 shards). That number fixes how history ownership spreads across the cluster. It is not a dial. Changing it requires a database reset³, which means export, rebuild, and reimport, planned like a migration with its own runbook and rollback page.

This bites exactly when growth feels good. Traffic doubles. Latency on history writes climbs. The scaling guide names the cause directly: the persistence backend becomes the bottleneck¹. The team opens the config expecting a larger number and finds a rebuild instead.

Consider, for example, a cluster sized for a pilot that now carries production. The fix the team wants is one edited field. The fix the engine requires is a weekend. Planning covers the export window, the verification pass, and the rollback decision (who calls it, and at what error rate). None of that work serves customers.

A static shard count also freezes past guesses into present capacity. A count chosen when histories were short governs a fleet whose histories are now long. The number cannot learn.

Worse, the reset punishes the workflows that can least afford it. Long-lived executions in flight during a rebuild need mapping from old shards to new ones, verified link by link. A fleet with, for example, thousands of open executions turns the migration into a reconciliation project with its own headcount. Short-lived fleets bounce. Yours negotiates, which is why the shard decision made earliest costs the most later.

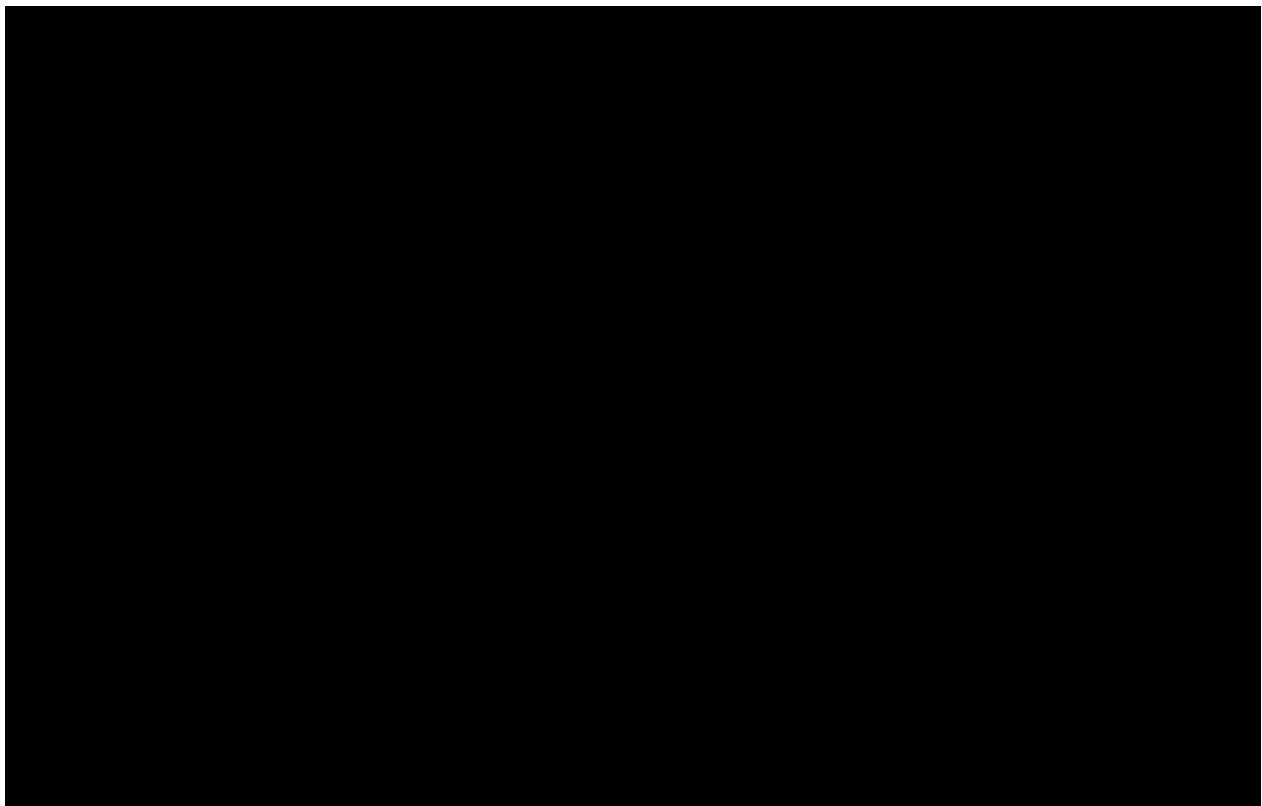


Figure 2: Deployment shapes side by side: fixed shard count versus versioned deployments.

Capacity planning under a fixed shard count rewards caution over ambition, so growth arrives as a migration project with a freeze window, which means the engine comfortable at one size charges a rebuild at the next.

4. Continue-As-New keeps long workflows alive by making your team hand-partition what the engine cannot hold.

Histories have a ceiling. The Continue-As-New docs set it plainly: 51,200 events or 50 MB per execution as a hard limit, with a warning at 10,240 events⁴. Payloads add their own walls nearby: 2 MB per payload and 4 MB per gRPC message⁵. Long workflows hit these walls by succeeding, one event at a time.

Continue-As-New (the API that closes the current execution and starts a successor with fresh history) is the documented answer. The developer chooses the cut point, copies the needed state into the new run, and chains executions forward. It works. Teams run chains of, for example, dozens of continuations for workflows that live past a year.

The charge is design labor the engine declines. Your team decides where one execution ends and the next begins. Your team writes the state-carry code. Your team debugs the chain when, for example, one link in a thirty-link run carries a field the next misreads. The engine holds each link; nobody holds the chain except your code.

Signals and queries complicate the partition further. A signal addressed to a closed execution must find the live successor. A query over the full workflow must span links the engine stores as separate executions. Both patterns are solvable. Both are yours to solve.

Upgrades inherit the same seam. A code change that lands mid-chain must deploy across executions that each carry their own history and version markers, so one release touches, for example, the partition logic, the state-carry schema, and the signal-forwarding map at once. Three surfaces. One deploy. Any skew between them surfaces as a stuck chain discovered days later, when the successor that never started pages nobody.

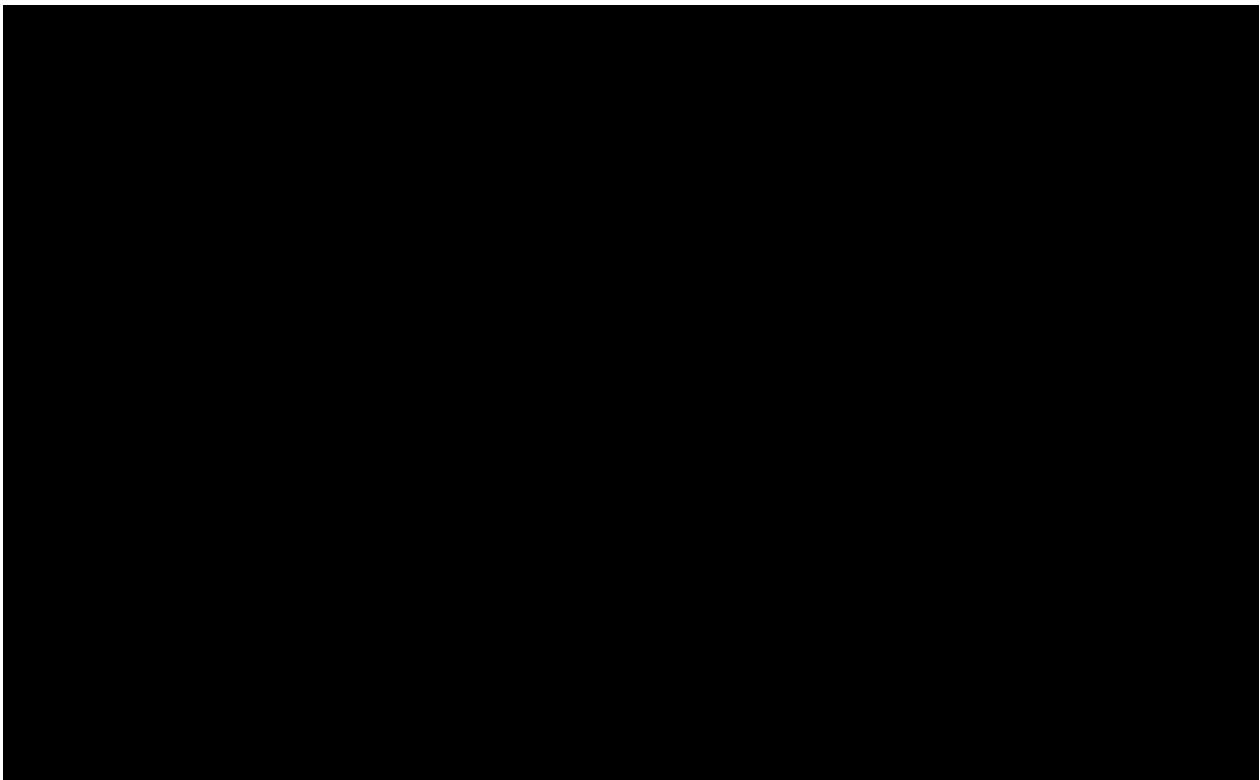


Figure 3: Continue-As-New chain: one long workflow becomes many short executions the team must link.

Continue-As-New converts one long workflow into many short ones and hands your team the scissors, so longevity ships as application code the team owns forever, which means the engine stays healthy by moving the hard part across the API boundary onto you.

5. Restate pushes work to versioned deployments, so a new release drains the old one instead of branching inside your workflow code.

Restate inverts the delivery path. Temporal workers poll task queues; Restate pushes work to registered deployments over a bidirectional SDK connection (a persistent channel the SDK opens to the runtime)⁶. Registration is explicit: a GET /services call on the Admin API returns each service with its handlers, deployment_id, and revision⁷. The runtime knows exactly which binary serves which handler at which revision.

That registry entry is what makes releases boring. A new deployment registers under a new deployment_id. In-flight invocations finish on the old revision. New invocations route to the new one. No markers accumulate in handler code because the old code keeps running until its work drains, which means the release history lives in the deployment list instead of in branches inside the workflow file.

The packaging story matches. Restate ships as a single binary in converged mode (runtime, metadata store, and user-space services colocated for small setups)⁸, so the team testing the version-drain behavior on a laptop runs the same binary shape that stages it. Push replaces the poll loop. Versioning replaces the marker.

Rollback follows the same path in reverse. The old deployment stays registered until its drain completes, which means a bad release reverts by routing new invocations back, not by reverting code and re-marking branches. Two revisions. One routing change. Recovery measured in minutes reads differently from recovery measured in replays, which is the operational gap the trial in the next section prices.

One claim in this section carries a label. Restate's lack of serialization limits (handlers passing objects without the payload ceilings Temporal documents) is owner-attested from production use, not publicly sourced. Treat it as a field report, stated here so a trial can confirm or refute it directly.

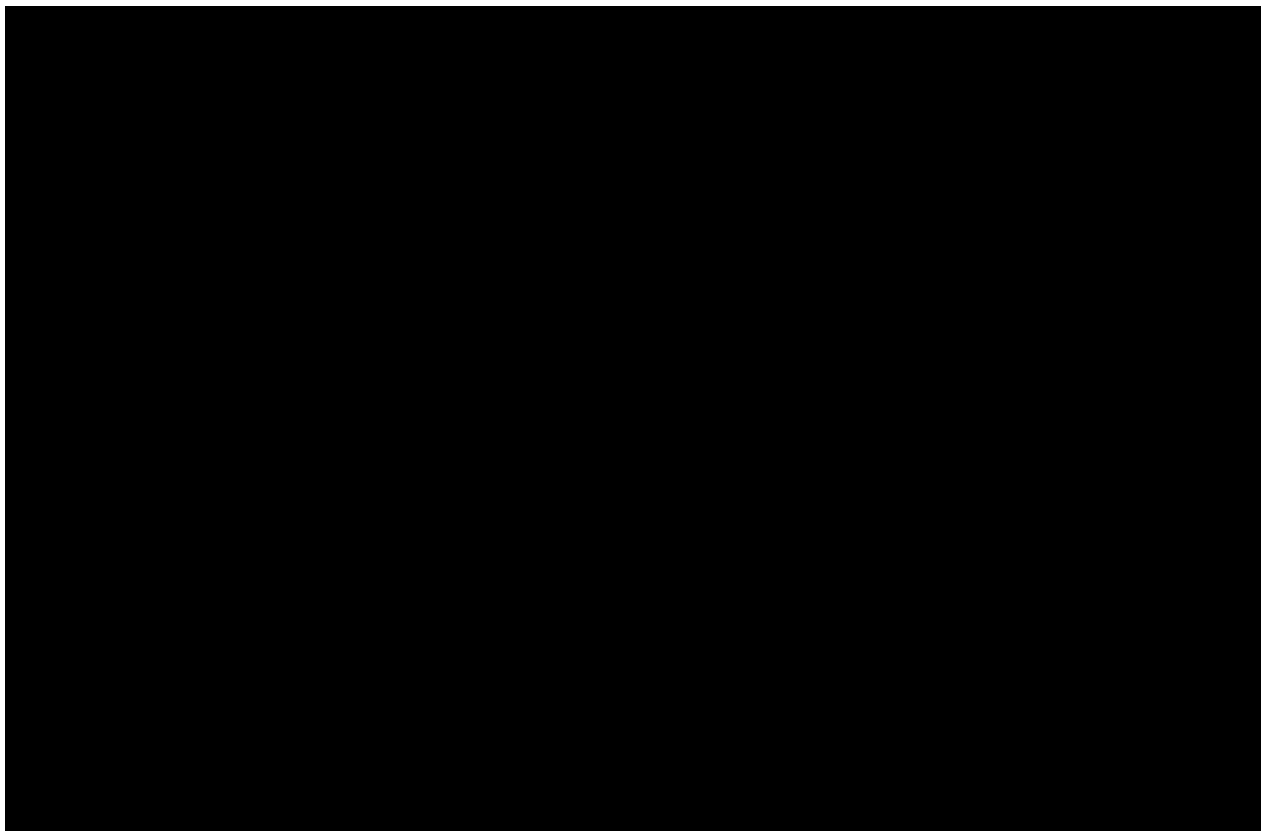


Figure 4: Failure timeline, two lanes: routing back to a draining revision versus replaying branches.

A release becomes two running revisions and a draining counter, so the workflow file stays at one version while the platform holds both, which means the team deletes old deployments instead of maintaining old branches.

6. Run one long-lived workflow on both engines past one release, and let the version diff decide.

Pick the workflow that already hurts. It runs for months. It carries real state. It will survive at least one release mid-flight. Port it straight, without redesigning around either engine's strengths, and run both copies against the same load for, for example, a single quarter.

Watch four named things. First, the version diff: count the branches the Temporal copy adds per release against the deployment entries the Restate copy adds. Second, history size: track event counts toward the 10,240 warning and the 51,200 cap on the Temporal side⁴. Third, release labor: log the hours spent on markers, version sets, continuation boundaries, and shard review. Fourth, recovery behavior: kill a worker mid-run on each side and time the resume.

Two mechanics deserve attention during the trial. Idempotency-key attach (a client-supplied key Restate holds for 24 hours so retries rejoin the original invocation)⁹ covers the duplicate-submit cases Temporal teams often solve with workflow IDs. Exactly-once delivery in Restate is owner-attested from production use, stated here under that label until the trial confirms it; score it as observed or not, not as doctrine.

Judge on the diff, not the demo. If the Temporal copy ships its release with two markers and one continuation boundary while the Restate copy ships with a second deployment that drains, the cost gap is concrete. Numbers from your own workflow beat any paper's verdict.

Scope the trial so it finishes. One workflow, one release, one quarter, for example, with a named owner and a written stop rule (the trial ends when the release drains on both sides, not when opinions settle). Timebox the instrumentation to a week. Score the four measures in a single page the whole team reads. Small trials conclude. Grand evaluations stall, which is why this one fits inside a planning cycle.

Run the trial past exactly one release, then read the two diffs side by side and fund the smaller one, which means the decision rests on artifacts your team produced rather than claims either vendor printed.

Notes

1 Temporal scaling guide — dev.to/temporalio/scaling-temporal-the-basics-3115

2 Temporal worker versioning — temporal.io/blog/automated-worker-versioning-with-github-actions

3 Temporal shard configuration — medium.com/@yedidya.s/from-step-functions-to-temporal-on-eks...

4 Temporal Continue-As-New — docs.temporal.io/develop/ruby/workflows/continue-as-new

5 Temporal payload limits — docs.temporal.io/troubleshooting/blob-size-limit-error

6 Restate runtime — restate.dev/blog/building-a-modern-durable-execution-engine-from-first-principles

7 Restate Admin API — docs.restate.dev/admin-api/service/list-services

8 Restate converged mode — restate.dev/blog/building-a-modern-durable-execution-engine-from-first-principles

9 Restate idempotency — docs.restate.dev/services/invocation/http